



ASSIGNMENT: 04

Submitted By: Manahil Kamran

Reg.No.: FA24-BSE-025

Submitted To: Ma'am Ambreen Gul

Date: 2nd May, 2026

Course: Information Security

Table Of Content

Task 1: Simple RSA Implementation (Without Libraries)	3
1. Introduction	3
2. Line- by- Line Explanation	3
Key Generation	3
Encryption	3
Decryption	4
Test Run	4
3. Security Analysis	4
Strengths:	4
Weaknesses:	4
4. Screenshots of Code:	5
Output:	6
Task 2: Create a Digital Signature	7
1. Introduction	7
2. Line- by- Line Explanation	7
Key Pair	7
Hashing	7
Signing	7
Verification	7
Tampering Test	7
3. Security Analysis	8
Strengths:	8
Weaknesses:	8
4. Screenshots Of Code:	8
Code:	8
Output:	9

Task 1: Simple RSA Implementation (Without Libraries)

Objective: Understand the math behind RSA Tasks:

1. Introduction

RSA is a public-key cryptosystem based on the difficulty of factoring large integers. It uses two keys: a public key for encryption and a private key for decryption. This Python implementation demonstrates RSA key generation with small primes, encryption, and decryption.

2. Line- by- Line Explanation

Key Generation

```
p = 61 # First prime
q = 53 # Second prime
n = p * q # Modulus
phi = (p-1)*(q-1) # Euler's totient
Choose two small primes p and q.
Compute modulus n.
Compute totient phi.
e = 17 # Public exponent (coprime with phi)
Choose e such that gcd(e, phi) = 1.
def modinv(a, m):
    # Extended Euclidean Algorithm to find modular inverse
    def egcd(a, b):
        if a == 0: return (b, 0, 1)
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)
    g, x, y = egcd(a, m)
    if g != 1: raise Exception('No modular inverse')
    return x % m

d = modinv(e, phi) # Private exponent
Compute modular inverse of e modulo phi to get private key d.
```

Encryption

```
def encrypt(msg, e, n):
    return [pow(ord(c), e, n) for c in msg]
Convert each character to ASCII (ord).
Raise to power e modulo n.
```

Decryption

```
def decrypt(cipher, d, n):  
    return ''.join([chr(pow(c, d, n)) for c in cipher])
```

**Raise each ciphertext number to power d modulo n.
Convert back to characters (chr).**

Test Run

```
message = "Manahil"  
cipher = encrypt(message, e, n)  
print("Ciphertext:", cipher)  
decrypted = decrypt(cipher, d, n)  
print("Decrypted:", decrypted)
```

Encrypt and decrypt a short string.

3. Security Analysis

Strengths:

- Public/private key separation.
- Based on hard problem of factoring large numbers.

Weaknesses:

- Small primes are insecure.
- Real RSA uses 2048+ bit keys.
- Use Cases: Secure communication, key exchange.

4. Screenshots of Code:

```
# Task 1: Simple RSA Implementation
# Objective: Understand the math behind RSA

# Step 1: Choose two small prime numbers
p = 61
q = 53

# Step 2: Compute modulus n and Euler's totient phi
n = p * q
phi = (p - 1) * (q - 1)

# Step 3: Choose public exponent e (must be coprime with phi)
e = 17

# Step 4: Compute private exponent d using modular inverse
def modinv(a: int, m: int) -> int: 1 usage
    # Extended Euclidean Algorithm
    def egcd(a: int, b: int) -> tuple:
        if a == 0:
            return (b, 0, 1)
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)
    g, x, y = egcd(a, m)
    if g != 1:
        raise Exception('No modular inverse')
    return x % m

d = modinv(e, phi)

print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
```

```
# Step 5: Encryption function
def encrypt(msg, e, n): 1 usage
    return [pow(ord(c), e, n) for c in msg]

# Step 6: Decryption function
def decrypt(cipher, d, n): 1 usage
    return ''.join([chr(pow(c, d, n)) for c in cipher])

# Step 7: Test with a short string (your name)
message = "Manahil"
cipher = encrypt(message, e, n)
print("Ciphertext:", cipher)

decrypted = decrypt(cipher, d, n)
print("Decrypted:", decrypted)
```

Output:

```
C:\Users\pc\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\pc\PyCharmMiscProject\Assig4.py
Public Key (e, n): (17, 3233)
Private Key (d, n): (2753, 3233)
Ciphertext: [3123, 1632, 2235, 1632, 2170, 3179, 745]
Decrypted: Manahil

Process finished with exit code 0
```

Task 2: Create a Digital Signature

Objective: Understand digital signatures Tasks:

1. Introduction

Digital signatures ensure authenticity and integrity of a message. A sender signs a message hash with their private key, and the receiver verifies it using the public key. This implementation shows how modifying the message invalidates the signature.

2. Line- by- Line Explanation

Key Pair

```
public_key = (e, n)
private_key = (d, n)
Public and private keys defined.
```

Hashing

```
import hashlib
message = "Manahil"
hash_val = int(hashlib.sha256(message.encode()).hexdigest(), 16)
Compute SHA256 hash of message.
Convert hex digest to integer.
```

Signing

```
signature = pow(hash_val, d, n)
Sign hash using private key.
```

Verification

```
verified_hash = pow(signature, e, n)
print("Valid signature?", verified_hash == hash_val % n)
Verify by raising signature to power e.
Compare with original hash.
```

Tampering Test

```
tampered_message = "Manahil!"
tampered_hash = int(hashlib.sha256(tampered_message.encode()).hexdigest(), 16)
print("Valid after tampering?", pow(signature, e, n) == tampered_hash % n)
Slightly modify message.
Verification fails.
```

3. Security Analysis

Strengths:

- Ensures authenticity and integrity.
- Detects tampering.

Weaknesses:

- Relies on secure hash functions.
- Small modulus insecure.
- Use Cases: Document signing, certificates, blockchain transactions.

4. Screenshots Of Code:

Code:

```
# Task 2: Digital Signature with RSA
# Objective: Understand how digital signatures work

import hashlib # Import hashlib for SHA256 hashing

# --- RSA Key Generation (reuse from Task 1) ---
p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17

# Extended Euclidean Algorithm to compute modular inverse
def modinv(a : {__eq__, __mod__, __floordiv__}, m : {__mod__, __floordiv__}): 1 usage
    def egcd(a : {__eq__, __mod__, __floordiv__}, b : {__mod__, __floordiv__}):
        if a == 0:
            return (b, 0, 1)
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)
    g, x, y = egcd(a, m)
    if g != 1:
        raise Exception('No modular inverse')
    return x % m

d = modinv(e, phi)

# Define public and private keys
public_key = (e, n)
private_key = (d, n)

print("Public Key:", public_key)
```

```

print("Public Key:", public_key)
print("Private Key:", private_key)

# --- Step 1: Create a message and compute its hash ---
message = "Manahil"
hash_val = int(hashlib.sha256(message.encode()).hexdigest(), 16) # Convert hash to integer

print("\nOriginal Message:", message)
print("SHA256 Hash (integer):", hash_val)

# --- Step 2: Sign the hash using the private key ---
signature = pow(hash_val, d, n) # Signature = hash^d mod n
print("Digital Signature:", signature)

# --- Step 3: Verify the signature using the public key ---
verified_hash = pow(signature, e, n) # Recover hash using public key
print("Signature Valid?", verified_hash == hash_val % n)

# --- Step 4: Tampering demonstration ---
tampered_message = "Manahil!" # Slightly modified message
tampered_hash = int(hashlib.sha256(tampered_message.encode()).hexdigest(), 16)

print("\nTampered Message:", tampered_message)
print("Tampered Hash (integer):", tampered_hash)
print("Signature Valid After Tampering?", pow(signature, e, n) == tampered_hash % n)

```

Output:

```

C:\Users\pc\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\pc\PyCharmMiscProject\Assign4_2.py
Public Key: (17, 3233)
Private Key: (2753, 3233)

Original Message: Manahil
SHA256 Hash (integer): 35417825418586616110982244620434751518155252830449097345956778976899075430691
Digital Signature: 2499
Signature Valid? True

Tampered Message: Manahil!
Tampered Hash (integer): 11426297066153361659223235729407590281405796303457285516314297206460006076299
Signature Valid After Tampering? False

Process finished with exit code 0

```